

Jaire Bartlett

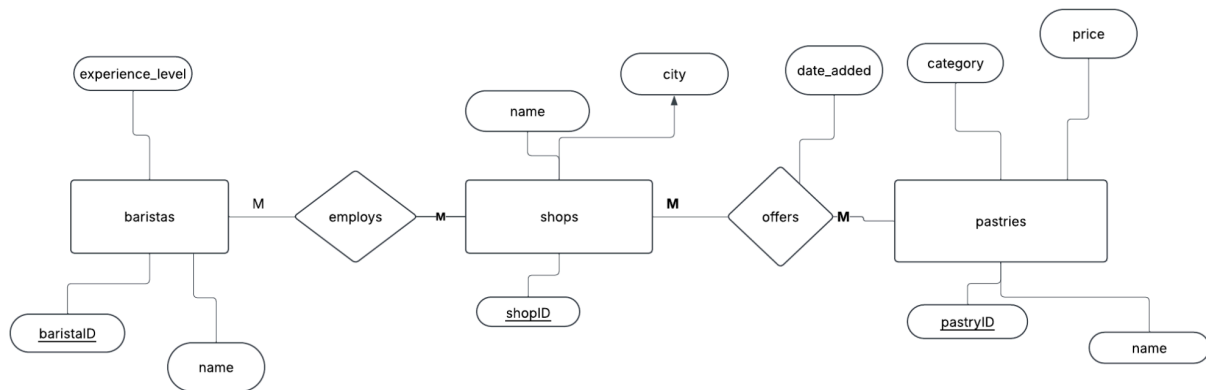
CSC 351

22 September 2026

Homework Assignment 2

The Entity-Relationship (ER) Model & Relational Mapping

Full Diagram:



Relational Tables:

- 1.) **baristas**(baristaID, name, experience_level) → baristaID is the Primary Key
- 2.) **shops**(shopID, name, city) → shopID is the Primary Key
- 3.) **pastries**(pastryID, name, category, price) → pastryID is the Primary Key
- 4.) **employs**(baristaID, shopID) → baristaID & shopID are Foreign Keys, collectively form composite primary key
- 5.) **offers**(shopID, pastryID, date_added) → shopID & pastryID are Foreign Keys

Participation & Cardinality Notes:

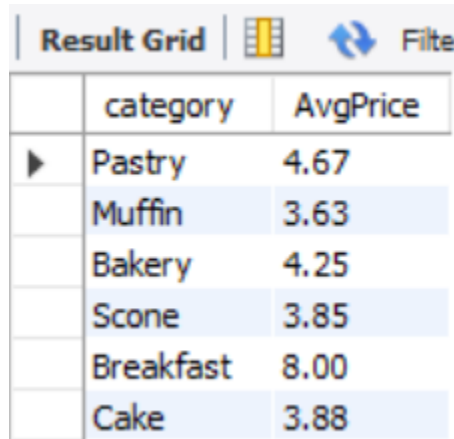
- 1.) In the relationship *employs* the Entity Sets *baristas* and *shops* have partial participation. Nothing in the creation of our SQL script makes the relationships total participation. The cardinality of the relationship is Many to Many because many baristas can be employed at many different shops and many shops can employ multiple baristas.
- 2.) In the relationship *offers* the cardinality is Many to Many because many shops can offer many pastries and many pastries can be offered at different shops. The relationship participation is partial for similar reasons listed above for the *employs* relationship.

SQL Queries & Screenshots

SQL Queries:

1. Find the average price of pastries for each category from the pastries table.

a. `select category, round(avg(price), 2) as 'AvgPrice'`
`from pastries`
`group by category;`

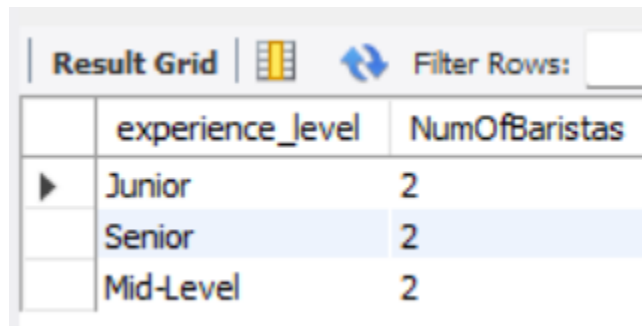


The screenshot shows a 'Result Grid' window with a table containing two columns: 'category' and 'AvgPrice'. The table has seven rows of data. The categories are Pastry, Muffin, Bakery, Scone, Breakfast, and Cake. The average prices are 4.67, 3.63, 4.25, 3.85, 8.00, and 3.88 respectively. The 'Filter' button is visible in the top right corner.

	category	AvgPrice
▶	Pastry	4.67
	Muffin	3.63
	Bakery	4.25
	Scone	3.85
	Breakfast	8.00
	Cake	3.88

2. Find the total number of baristas at each experience level from the baristas table.

a. `select experience_level, count(*) as 'NumOfBaristas'`
`from baristas`
`group by experience_level;`



The screenshot shows a 'Result Grid' window with a table containing two columns: 'experience_level' and 'NumOfBaristas'. The table has three rows of data. The experience levels are Junior, Senior, and Mid-Level. The number of baristas for each level is 2. The 'Filter Rows' button is visible in the top right corner.

	experience_level	NumOfBaristas
▶	Junior	2
	Senior	2
	Mid-Level	2

3. Count the total number of shops located in each city from the shops table.

a. `select city, count(shopID) as 'NumOfShops'`
`from shops`
`group by city;`

Result Grid			Filter Row
	city	NumOfShops	
▶	Seattle	1	
	Portland	1	
	Austin	1	
	Chicago	1	
	Denver	1	
	Boston	1	

4. Find the maximum price among pastries for each category from the pastries table.

- a. `select category, max(price) as 'MaxPrice'`
`from pastries`
`group by category;`

Result Grid			Filter
	category	MaxPrice	
▶	Pastry	5.50	
	Muffin	3.75	
	Bakery	4.25	
	Scone	3.85	
	Breakfast	8.50	
	Cake	4.00	

5. Count how many pastries have been added by each shop using the shopID column from the offers table.

- a. `select shopID, count(pastryID) as 'NumOfPastries'`
`from offers`
`group by shopID;`

Result Grid			Filter Rows
	shopID	NumOfPastries	
▶	1	2	
	2	2	
	3	2	
	4	2	
	5	2	
	6	2	

6. Find the name, category, and price of any pastry whose price matches the maximum price within its category.

a. `select` name, category, price
`from` pastries
 where (category, price) `in` (
 `select` category, `max`(price)
 `from` pastries
 `group by` category
);

Result Grid				Filter Rows:
	name	category	price	
▶	Almond Croissant	Pastry	5.50	
	Blueberry Muffin	Muffin	3.75	
	Cinnamon Roll	Bakery	4.25	
	Chocolate Scone	Scone	3.85	
	Blueberry Scone	Scone	3.85	
	Breakfast Burrito	Breakfast	8.50	
	Lemon Loaf	Cake	4.00	

7. Find the unique shop IDs from the offers table that have offered at least one pastry whose price is strictly greater than the overall average price of all pastries.

a. `select distinct` shopID
`from` offers
`where` pastryID `in` (

```

select pastryID
from pastries
where price > (select avg(price) from pastries)
);

```

Result Grid	
	shopID
▶	1
	5

8. Find the shop ID and pastry ID for the records in the offers table that have the earliest date_added (minimum date).

a.

```

select shopID, pastryID
from offers
where date_added = (select min(date_added) from offers);

```

Result Grid		
	shopID	pastryID
▶	1	1
✱	NULL	NULL

9. Find the shop ID(s) that offer the highest number of pastries, utilizing a subquery to evaluate the maximum count per shop.

a.

```

select shopID, count(pastryID) as 'NumOfPastries'
from offers
group by shopID
having count(pastryID) >= all (
    select count(pastryID)
    from offers
    group by shopID
);

```

Result Grid



Filter Rows

	shopID	NumOfPastries
▶	1	2
	2	2
	3	2
	4	2
	5	2
	6	2

10. Find the names of baristas who work at shops located in 'Seattle' using nested subqueries.

- a. `select name`
`from baristas`
`where baristaID in (`
`select baristaID`
`from employs`
`where shopID in (`
`select shopID`
`from shops`
`where city = 'Seattle'`
`)`
`);`

Result Grid

	name
▶	Emma Watson
	Liam Chen